2 - Encapsulamiento

Tabla de contenidos

1 Prefacio 1
2 Introducción 2
3 Las múltiples caras del encapsulamiento 2
  3.1 Funciones 2
  3.2 Objetos 3
4 Interfaz clara 3
  4.1 *Docstrings* 3
  4.2 Anotaciones de tipo 7
5 Estado interno 9
  5.1 *Setters* y *getters* 10
  5.2 Atributos protegidos 12
  5.3 Atributos privados 13
  5.4 Atributos (aparentes) con `@property` 15
  5.5 Resumen 17
6 Atributos y métodos de clase 17
  6.1 Atributos 17
  6.2 Métodos 19

1 Prefacio

Los tres principios fundamentales de la programación orientada a objetos son la **encapsulación**, la **herencia** y el **polimorfismo**.

A lo largo de los próximos apuntes vamos a explorar cada uno en detalle, entendiendo los conceptos que los sustentan y analizando ejemplos concretos de cómo se aplican en Python. En líneas generales, estos principios se pueden describir de la siguiente manera:

- **Encapsulación:** consiste en reunir en un mismo lugar tanto los datos como las operaciones que actúan sobre ellos, ocultando los detalles internos y exponiendo únicamente la interfaz necesaria para interactuar con el objeto.
- **Herencia:** permite crear nuevas clases a partir de otras ya existentes, reutilizando su comportamiento y ampliándolo o modificándolo según sea necesario.
- **Polimorfismo:** hace posible que diferentes objetos de diferentes clases respondan de forma distinta a un mismo "mensaje" (por ejemplo, un método con el mismo nombre), adaptando el comportamiento a las particularidades de cada caso.

2 Introducción

Supongamos que nos encontramos manejando un auto por el centro rosarino y al llegar a la esquina vemos que por la calle perpendicular se aproxima otro vehículo que no aparenta intenciones de frenar.

Todo indica que tendremos que detener el auto completamente.

Instantáneamente, presionamos el embrague casi al mismo tiempo que el pedal de freno y colocamos la palanca de cambio en la posición de punto muerto. El auto responde de la manera que esperamos y se detiene.

Una vez que el otro vehículo cruza, nos disponemos a continuar nuestra marcha. Como aún no soltamos el pie del embrague, movemos la palanca de cambios a la posición de primera, suavemente soltamos el embrague mientras comenzamos a presionar el acelerador, y finalmente cruzamos.

¿Y qué tiene que ver toda esta escena automovilística con el encapsulamiento? Más de lo que podríamos imaginarnos.

Para detener el auto, tuvimos que interactuar con los pedales y eventualmente con la palanca de cambios. Todo un esfuerzo, sí.

Sin embargo, no necesitamos saber en realidad como funciona el proceso de frenado de un auto: desconocemos como funcionan los discos, la hidráulica y mucho menos podríamos describir como funciona una caja de cambios. Todos estos mecanismos internos permanecen ocultos dentro del sistema (el auto). Lo único visible es una interfaz sencilla que nos permite lograr nuestro objetivo sin necesidad de saber qué ocurre detrás.

En programación ocurre lo mismo: la encapsulación consiste en mantener el estado interno y la lógica de un objeto fuera del alcance del exterior, exponiendo únicamente una forma clara y controlada de interactuar con él. De este modo, el código que interactua con el objeto no necesita conocer sus detalles internos y puede seguir funcionando incluso si estos cambian.

3 Las múltiples caras del encapsulamiento

En programación no existe una única manera de aplicar el encapsulamiento, es decir, de aislar y proteger los detalles internos de cómo algo funciona. A continuación, veremos cómo esta idea aparece y se utiliza en distintos niveles: funciones, objetos y clases.

3.1 Funciones

Las funciones ofrecen un ejemplo clarísimo de encapsulación: para usarlas, no hace falta conocer como funcionan internamente. De hecho, una función bien diseñada se caracteriza por cumplir una única tarea y tener un nombre que la describa con claridad. De esta manera, con solo leer su nombre podemos anticipar qué hace, sin preocuparnos por los detalles de su implementación.

Por ejemplo, consideremos la siguiente función que devuelve el número n-ésimo en la secuencia de Fibonacci:

```python
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
print(fibonacci(10))
print(fibonacci(21))
```

```
55
10946
```

Si está clara la interfaz de la función (cantidad y tipos de entradas y salidas), no es necesario conocer detalles de la implementación, ni si el código es largo o complejo. Incluso si se encuentra un mejor algoritmo para resolver el mismo problema, la función puede reescribirse sin cambiar su uso externo, siempre que la interfaz no cambie.

Esta modularización hace que el código sea más fácil de mantener y adaptar a futuros cambios.

3.2 Objetos

En la programación orientada a objetos hay una distinción clave entre el *interior* y el *exterior* de una clase u objeto.

Desde el **interior**, al diseñar una clase o implementar sus métodos, debemos cuidar cómo interactúan con los atributos, la eficiencia de los algoritmos y el diseño de la interfaz. El objetivo es construir una estructura coherente y fácil de mantener.

Desde el **exterior**, lo que importa no son los detalles internos sino la interfaz pública: qué hace cada método, qué argumentos necesita y qué valores devuelve. Mientras esa interfaz se mantenga, la clase puede usarse sin conocer su implementación.

Las clases favorecen el encapsulamiento porque:

- Definen una interfaz clara para usar sus métodos sin saber cómo funcionan por dentro.
- Protegen el estado interno, evitando modificaciones directas desde fuera.

Y, gracias a ello, permiten cambiar la implementación sin afectar el código que las utiliza.

Ahora bien, ¿cómo es que las clases en Python definen una interfaz clara y protegen el estado interno?

4 Interfaz clara

4.1 *Docstrings*

¿Cómo podemos saber cuántos argumentos debemos pasar y de qué tipo al inicializar una clase? ¿Y cómo saberlo al llamar a uno de sus métodos?

Una primera opción sería leer directamente su implementación, pero eso es precisamente lo que queremos evitar.

Una alternativa mucho mejor es consultar la documentación. Para ello, la clase y sus métodos deben contar con *docstrings* adecuados que describan su uso.

Veamos el siguiente ejemplo:

```
class CuentaBancaria:
    """Cuenta bancaria simple con operaciones básicas.

    Esta clase implementa un modelo básico de cuenta bancaria que permite
    depositar y retirar dinero, así como consultar el saldo actual.
    """
    def __init__(self, titular, saldo_inicial=0.0):
        """Inicializa una nueva cuenta bancaria.

        Parameters
        ----------
        titular : str
            Nombre del titular de la cuenta.
        saldo_inicial : float, optional
            Saldo inicial de la cuenta. Por defecto es 0.0.
        """
        self.titular = titular
        self.saldo = saldo_inicial

    def depositar(self, monto):
        """Depositar dinero en la cuenta.

        Parameters
        ----------
        monto : float
            Monto a depositar. Debe ser un número positivo.
        """
        if monto <= 0:
            print("Error: El monto a depositar debe ser positivo.")
            return
        self.saldo += monto


    def retirar(self, monto):
        """Extrae dinero de la cuenta si hay fondos suficientes.

        Parameters
        ----------
        monto : float
            Monto a retirar.
        """
        if monto > self.saldo:
            print("Error: Fondos insuficientes.")
            return
```

```
        self.saldo -= monto


    def consultar_saldo(self):
        """Devuelve el saldo actual.

        Returns
        -------
        float
            Saldo disponible en la cuenta.
        """
        return self.saldo
```

```
cuenta = CuentaBancaria(titular="José Paso", saldo_inicial=12000)
cuenta.consultar_saldo()
```

```
12000
```

```
cuenta.retirar(3800)
cuenta.consultar_saldo()
```

```
8200
```

```
cuenta.depositar(1500)
cuenta.retirar(50000)
cuenta.consultar_saldo()
```

```
Error: Fondos insuficientes.

9700
```

Para consultar la documentación, podemos usar la función `help` de Python.

Si la ejecutamos en Positron, se abrirá una ventana a la derecha que muestra la información disponible. Si, en cambio, la usamos desde una terminal, obtendremos un resultado similar al siguiente:

Ayuda para toda clase:

```
help(CuentaBancaria)
```

```
Help on class CuentaBancaria in module __main__:
```

```
class CuentaBancaria(builtins.object)
 |  CuentaBancaria(titular, saldo_inicial=0.0)
 |
 |  Cuenta bancaria simple con operaciones básicas.
 |
 |  Esta clase implementa un modelo básico de cuenta bancaria que permite
 |  depositar y retirar dinero, así como consultar el saldo actual.
 |
 |  Methods defined here:
 |
 |  __init__(self, titular, saldo_inicial=0.0)
 |      Inicializa una nueva cuenta bancaria.
 |
 |      Parameters
 |      ----------
 |      titular : str
 |          Nombre del titular de la cuenta.
 |      saldo_inicial : float, optional
 |          Saldo inicial de la cuenta. Por defecto es 0.0.
```

Ayuda para el método `depositar`, que recibe un flotante y no devuelve nada:

```
help(CuentaBancaria.depositar)
```

```
Help on function depositar in module __main__:

depositar(self, monto)
    Depositar dinero en la cuenta.

    Parameters
    ----------
    monto : float
        Monto a depositar. Debe ser un número positivo.
```

Ayuda para el método `consultar_saldo`, que no recibe ningún parametro y devuelve un flotante:

```
help(CuentaBancaria.consultar_saldo)
```

```
Help on function consultar_saldo in module __main__:

consultar_saldo(self)
    Devuelve el saldo actual.

    Returns
    -------
```

```
    float
        Saldo disponible en la cuenta.
```

Si bien en algunos casos puede resultar necesario consultar la ayuda con la función `help`, los editores de código suelen mostrar automáticamente una pequeña ventana junto al código mientras escribimos, donde aparece la documentación de clases, métodos y funciones.

4.2 Anotaciones de tipo

Si bien Python es un lenguaje de **tipado dinámico** —es decir, no es necesario declarar el tipo de las variables y una variable puede referenciar objetos de distintos tipos durante la ejecución—, es posible utilizar **anotaciones de tipo** (del inglés *type annotations*) para indicar qué tipo de dato se espera. Estas anotaciones son opcionales, pero ayudan a que el código sea más claro, fácil de entender y detectar errores antes de ejecutar el programa.

Para los parámetros de una función o método, las anotaciones de tipo se escriben con el formato `<nombre_variable>: <tipo>`. En el caso la salida, el tipo se indica después de una flecha (`-> <tipo>`) al final de la definición.

Por ejemplo, en la siguiente función se especifica que el parámetro `nombre` debe ser de tipo `str` y que la función devuelve también un objeto de tipo `str`:

```
def saludar(nombre: str) -> str:
    return f"¡Hola, {nombre}!"
saludar("Guido")
```

```
'¡Hola, Guido!'
```

En nuestra clase `CuentaBancaria`, la anotación de tipos se ve de la siguiente manera:

```
class CuentaBancaria:
    def __init__(self, titular: str, saldo_inicial: float = 0.0):
        self.titular = titular
        self.saldo = saldo_inicial

    def depositar(self, monto: float):
        self.saldo += monto

    def retirar(self, monto: float):
        self.saldo -= monto

    def consultar_saldo(self) -> float:
        return self.saldo
```

Línea 2

Para inicializar la clase, se espera un argumento `titular` de tipo `str` y otro `saldo_inicial` de tipo `float`.

Línea 6

El método `depositar` espera recibir un argumento de tipo `float`.

Línea 9

El método `retirar` también espera recibir un argumento de tipo `float`.

Línea 12

Por último, `consultar_saldo` devuelve un valor de tipo `float`.

Si se eligen nombres representativos para los métodos y se utilizan anotaciones de tipo en sus parámetros, es muy probable que no sea necesario escribir un *docstring* para que el usuario comprenda cómo funciona la clase.

Sin embargo, ni la documentación mediante *docstrings* ni el uso de anotaciones de tipo garantizan que una función o método se utilice con los tipos de datos adecuados. Por ejemplo, podríamos pasarle un número a nuestra función `saludar` sin que Python lo impida:

```
saludar(128)
```

```
'¡Hola, 128!'
```

O incluso podríamos inicializar el `saldo_inicial` de la cuenta bancaria con una lista y luego intentar "depositar" otra lista.

```
cuenta = CuentaBancaria(25, saldo_inicial=["Cosas"])
cuenta.consultar_saldo()
```

```
['Cosas']
```

```
cuenta.depositar(["Otras cosas", "Aún más cosas"])
cuenta.consultar_saldo()
```

```
['Cosas', 'Otras cosas', 'Aún más cosas']
```

En otras palabras, si bien Python nos permite especificar la interfaz de funciones y métodos mediante *docstrings* y anotaciones de tipo, al ser un lenguaje de **tipado dinámico** nada impide que se utilicen con tipos de datos para los que no fueron diseñados. En algunos casos esto puede resultar en comportamientos inesperados y, en otros, simplemente generar un error en tiempo de ejecución.

> ℹ *Duck typing* 🦆
>
> En lenguajes dinámicos como Python, muchas veces no importa de qué tipo es un objeto, sino qué puede hacer. Lo relevante no es su clase, sino si se comporta como necesitamos.
>
> Por ejemplo, en una función como `saludar`, el parámetro no tiene que ser necesariamente un `str`, siempre que pueda usarse dentro de una *f-string*.
>
> Este enfoque, donde importa más el comportamiento que el tipo, se llama *duck typing* y suele expresarse así:
>
> > Si camina como un pato y hace *cuac* como un pato, entonces probablemente es un pato.

5 Estado interno

Ahora que sabemos qué estrategias podemos usar para que la interfaz de una clase sea clara, veamos cómo las clases definen y protegen su estado interno.

Consideremos a la siguiente clase que sirve para representar a estudiantes de la Facultad de Ciencias Económicas y Estadística de la UNR.

```
class Estudiante:
    def __init__(self, nombre, ingreso, carrera):
        self.nombre = nombre
        self.ingreso = ingreso
        self.carrera = carrera

    def resumen(self):
        return f"Estudiante(nombre={self.nombre}, ingreso={self.ingreso},
carrera={self.carrera})"
```

Cada objeto mantiene sus propias variables de instancia, como `nombre`, `ingreso` y `carrera`, con valores independientes de los de otros objetos de la misma clase. Esto implica que modificar los datos de una instancia no afecta en absoluto a las demás: cada objeto gestiona y conserva su propio estado interno, es decir, los objetos son dueños de sus variables.

```
e1 = Estudiante("Mariano González", 2022, "Contador Público")
e2 = Estudiante("Leticia Gallardo", 2023, "Licenciatura en Estadística")
```

Gracias al método `resumen`, podemos obtener una representación clara e intuitiva de cada objeto.

```
e1.resumen()
```

```
'Estudiante(nombre=Mariano González, ingreso=2022, carrera=Contador Público)'
```

Aunque también es posible interactuar con los atributos de cada instancia de manera individual.

```
print(e1.nombre, e1.carrera, sep=": ")
print(e2.nombre, e2.carrera, sep=": ")
```

```
Mariano González: Contador Público
Leticia Gallardo: Licenciatura en Estadística
```

Esta interacción no solo implica que podemos acceder a los valores individuales de los atributos, sino también que tenemos la posibilidad de modificarlos.

```
e1.ingreso = 2019
e1.resumen()
```

```
'Estudiante(nombre=Mariano González, ingreso=2019, carrera=Contador Público)'
```

Tenemos tanta flexibilidad al modificar los atributos de una instancia que incluso podemos asignarles valores que no tienen sentido dentro del contexto de la clase.

```
e1.ingreso = "Cualquier cosa"
e1.resumen()
```

```
'Estudiante(nombre=Mariano González, ingreso=Cualquier cosa, carrera=Contador
Público)'
```

5.1 *Setters* y *getters*

En programación orientada a objetos, los *getters* y *setters* son métodos especiales que permiten **acceder** y **modificar** el estado interno de un objeto de forma segura y controlada. Su objetivo principal es proteger los atributos, evitando que se acceda o se cambien directamente desde el exterior de la clase.

En particular:

- *getter*: método que obtiene o devuelve el valor de un atributo de un objeto.
- *setter*: método que asigna o actualiza el valor de un atributo de un objeto.

Nuestra clase `Estudiante`, incorporando ahora estos métodos para acceder y modificar sus atributos, se vería de la siguiente manera:

```
class Estudiante:
    def __init__(self, nombre):
        self.setNombre(nombre)

    def setNombre(self, nombre):
```

```
        if isinstance(nombre, str):
            self.nombre = nombre
        else:
            print("El nombre debe ser de tipo 'str'")

    def getNombre(self):
        return self.nombre

    def resumen(self):
        return f"Estudiante(nombre={self.getNombre()})"
```

Línea 3

El método de inicialización no asigna el atributo directamente, sino que delega la tarea en el *setter*.

Línea 5

El *setter* recibe un valor, verifica su tipo y, si es el esperado, lo asigna como atributo de instancia.

Línea 11

El *getter* simplemente devuelve el valor del atributo, proporcionando un punto de acceso controlado al estado interno.

Creemos un nuevo objeto de tipo `Estudiante`.

```
e = Estudiante("Macarena Gianetti")
e.resumen()
```

```
'Estudiante(nombre=Macarena Gianetti)'
```

Luego, obtenemos el nombre de la estudiante mediante su *getter*.

```
e.getNombre()
```

```
'Macarena Gianetti'
```

Si queremos modificarlo, no asignamos el valor directamente a una variable del objeto, sino que llamamos a su método *setter*. Este método se encarga de validar el dato y evitar que se asignen valores de tipos no permitidos.

```
e.setNombre(189)
```

```
El nombre debe ser de tipo 'str'
```

```
e.resumen()
```

```
'Estudiante(nombre=Macarena Gianetti)'
```

Sin embargo, Python no impide que, como usuarios, accedamos y modifiquemos directamente los atributos del objeto. Por ejemplo, podemos asignar un valor de cualquier tipo directamente a la variable `nombre`:

```
e.nombre = 2
e.resumen()
```

```
'Estudiante(nombre=2)'
```

En ese caso, las ventajas de usar *setter* y *getter* dejan de tener efecto si el usuario decide "romper" el objeto ignorando su interfaz.

5.2 Atributos protegidos

En muchos lenguajes de programación existe una distinción clara entre atributos **públicos** y **privados**. Los públicos pueden ser accedidos tanto desde dentro como desde fuera de la clase, mientras que los privados solo pueden usarse internamente. Es decir, un método de la clase puede acceder a un atributo o método privado, pero el código externo que usa la clase no.

En **Python** no existe una separación estricta entre atributos públicos y privados: **todos son técnicamente públicos**. Sin embargo, por **convención**, si el nombre de un atributo o método comienza con un guion bajo (_), esto indica que no debería ser accedido ni modificado desde el exterior de la clase, ya que está protegido.

Siguiendo esta convención, en nuestro ejemplo con la clase `Estudiante` podemos usar un atributo llamado `_nombre` para almacenar el nombre del estudiante.

```
class Estudiante:
    def __init__(self, nombre):
        self.setNombre(nombre)

    def setNombre(self, nombre):
        if isinstance(nombre, str):
            self._nombre = nombre
        else:
            print("El nombre debe ser de tipo 'str'")

    def getNombre(self):
        return self._nombre
```

```
    def resumen(self):
        return f"Estudiante(nombre={self.getNombre()})"
```

Línea 7

Se guarda el nombre en una variable "privada" `_nombre`.

Línea 12

Se devuelve el nombre usando la variable "privada" `_nombre`.

```
e = Estudiante("Macarena Gianetti")
e.resumen()
```

```
'Estudiante(nombre=Macarena Gianetti)'
```

Si asignamos un nuevo valor a la variable `nombre` de la instancia, no ocurre ningún efecto indeseado. El objeto crea y almacena esa nueva variable, pero el método *getter* no la utiliza, ya que sigue accediendo al atributo `_nombre`.

```
e.nombre = "Algo nuevo"
e.resumen()
```

```
'Estudiante(nombre=Macarena Gianetti)'
```

Sin embargo, Python no evita que reemplacemos el valor de la variable `_nombre`, nuevamente rompiendo el encapsulamiento del objeto:

```
e._nombre = "¡Ahora sí!"
e.resumen()
```

```
'Estudiante(nombre=¡Ahora sí!)'
```

5.3 Atributos privados

Si bien los objetos en Python no cuentan con atributos verdaderamente privados, es posible emular ese comportamiento. Para ello, se emplean nombres de variables que comienzan con dos guiones bajos (__). En nuestro ejemplo, podemos usar `__nombre`.

```
class Estudiante:
    def __init__(self, nombre):
        self.setNombre(nombre)

    def setNombre(self, nombre):
        if isinstance(nombre, str):
```

```
            self.__nombre = nombre
        else:
            print("El nombre debe ser de tipo 'str'")

    def getNombre(self):
        return self.__nombre

    def resumen(self):
        return f"Estudiante(nombre={self.getNombre()})"
e = Estudiante("Macarena Gianetti")
e.resumen()
```

```
'Estudiante(nombre=Macarena Gianetti)'
```

Si queremos acceder a la variable, obtendremos un error:

```
e.__nombre
```

```
AttributeError: 'Estudiante' object has no attribute '__nombre'
```

Por el contrario, si intentamos asignar un valor a esa variable, Python no mostrará ningún error y el método *getter* continuará devolviendo el valor esperado:

```
e.__nombre = "¿Y ahora?"
e.getNombre()
```

```
'Macarena Gianetti'
```

```
e._Estudiante__nombre
```

```
'Macarena Gianetti'
```

> **i** *Name mangling*
>
> Nunca debemos olvidar que Python no soporta atributos privados. Por lo tanto, en algún lado tiene que estar disponible el valor de la variable `__nombre` que se usa dentro de la clase.
>
> En particular, cuando el nombre de una variable comienza con dos guiones bajos, Python utiliza una técnica llamada *name mangling* o "estropeo de nombre". Para ello, en realidad opera internamente con otra variable, cuyo nombre es el resultado de concatenar un guión bajo, el nombre de la clase y el nombre del atributo "privado". Por ejemplo:
>
> ```
> e._Estudiante__nombre
> 'Macarena Gianetti'
> ```

5.4 Atributos (aparentes) con `@property`

Python ofrece un decorador *built-in* llamado `property` que permite definir un método que **se comporta como si fuera un atributo**, de modo que al accederlo desde fuera parece una variable de instancia, aunque en realidad está ejecutando código dentro de la clase.

Con este decorador se pueden definir dos métodos: un *getter* y un *setter*.

- El *getter* se declara con `@property`, y su nombre determina el nombre de la propiedad que se utilizará desde el código externo.
- El *setter* se declara con `@<nombre>.setter` y permite asignar valores a esa misma propiedad.

Veamos un ejemplo:

```
class Estudiante:
    def __init__(self, nombre):
        self.nombre = nombre

    @property
    def nombre(self):
        return self._nombre

    @nombre.setter
    def nombre(self, valor):
        if isinstance(valor, str):
            self._nombre = valor
        else:
            print("El nombre debe ser de tipo 'str'")

    def resumen(self):
        return f"Estudiante(nombre={self.nombre})"
```

Líneas 5-7

Con `@property` se indica que los objetos de la clase tendrán un "atributo" llamado `nombre`. Al acceder a él, Python ejecuta el método decorado y devuelve el valor de la variable protegida `_nombre`.

Líneas 9-14

Con `@nombre.setter` se declara el método *setter*, que recibe `self` y el nuevo valor a asignar (`valor`). Así, cada vez que se asigna un nuevo valor al atributo, Python ejecuta este método y verifica el tipo de dato.

Línea 3

Incluso dentro de la clase puede usarse `nombre` como si fuera un atributo común, sin necesidad de llamar manualmente al método decorado.

Línea 17

Ídem al punto anterior.

```
e = Estudiante("Fernanda Cattalini")
e.resumen()
```

```
'Estudiante(nombre=Fernanda Cattalini)'
```

Es posible acceder al "atributo" `nombre`:

```
e.nombre
```

```
'Fernanda Cattalini'
```

También modificarlo:

```
e.nombre = "María Fernanda Cattalini"
e.resumen()
```

```
'Estudiante(nombre=María Fernanda Cattalini)'
```

Y si se intenta asignarle un valor del tipo incorrecto, no se realiza la operación:

```
e.nombre = True
```

```
El nombre debe ser de tipo 'str'
```

```
e.resumen()
```

```
'Estudiante(nombre=María Fernanda Cattalini)'
```

5.5 Resumen

Python no ofrece un control absoluto sobre el estado interno de los objetos, pero sí dispone de mecanismos que permiten gestionarlo mejor, como las convenciones de nombres, los *getters* y *setters*, o el uso de `@property`. Estas herramientas ayudan a ocultar detalles internos, validar datos y mantener la coherencia del objeto.

Sin embargo, por la naturaleza dinámica del lenguaje, siempre existe la posibilidad de modificar clases y objetos desde el exterior, por lo que el encapsulamiento funciona más como una convención para un uso correcto que como una barrera estricta.

6 Atributos y métodos de clase

En Python, no solo los objetos pueden encapsular estado: las clases también.

Un **atributo de clase** está asociado a la clase en sí y es compartido por todas sus instancias, en lugar de pertenecer a un objeto específico.

Un **método de clase**, en cambio, recibe la propia clase como primer argumento (`cls`) en lugar de la instancia (`self`), lo que permite operar sobre la clase en su conjunto.

6.1 Atributos

Es posible asignar un atributo de clase en el código que implementa a la clase misma. Simplemente hay que asignar una variable en el bloque de defínción de la clase.

En el ejemplo a continuación, creamos una clase `Gato` que representa animales de la especie *Felis catus*. Como todos los gatos son de la misma especie, tiene sentido utilizar un atributo de clase en vez de un atributo de instancia.

```
class Gato:
    especie = "Felis catus"

    def __init__(self, nombre, raza=None):
        self.nombre = nombre
        self.raza = raza

    def resumen(self):
        return f"Gato(nombre={self.nombre}, raza={self.raza})"
```

Línea 2

Creación del atributo de clase `especie`.

Luego de instanciar dos objetos, podemos ver que ambos tienen asociados el mismo valor de `especie`.

```
g1 = Gato("Chispitas")
g2 = Gato("Bigotes", "Siamés")
```

```
print(g1.especie)
print(g2.especie)
print(g1.especie == g2.especie)
```

```
Felis catus
Felis catus
True
```

Podríamos utilizar un atributo de clase análogo para otra especie de animales: *Canis lupus familiaris*, popularmente conocidos como perro.

```
class Perro:
    especie = "Canis lupus familiaris"

    def __init__(self, nombre, raza=None):
        self.nombre = nombre
        self.raza = raza

    def resumen(self):
        return f"Perro(nombre={self.nombre}, raza={self.raza})"
```

```
perro = Perro("Bruno")
print(perro.resumen())
print(perro.especie)
```

```
Perro(nombre=Bruno, raza=None)
Canis lupus familiaris
```

Otro escenario donde tiene sentido práctico utilizar un atributo de clase es cuando se necesita mantener un estado global.

La clase `Usuario` define usuarios de una cierta aplicación. En ella, se tiene la variable `total_usuarios` que es un contador de los usuarios que se han creado a partir de la clase.

```
class Usuario:
    total_usuarios = 0

    def __init__(self, nombre):
        self.nombre = nombre
        Usuario.total_usuarios += 1
```

Línea 2

Inicialmente, el atributo de clase `total_usuarios` tiene el valor `0`.

Línea 6

Cada vez que se crea un nuevo usuario, se incrementa el valor del atributo de la clase `total_usuarios` en 1.

```
u1 = Usuario("Ana")
u2 = Usuario("Luis")
print(Usuario.total_usuarios)
```

```
2
```

Dado que las instancias también pueden acceder a los atributos de clase, se tiene:

```
print(u1.total_usuarios)
print(u2.total_usuarios)
```

```
2
2
```

6.2 Métodos

Para definir un método de clase se usa el decorador `@classmethod`, incluido en Python. Al aplicarlo, el método recibe como primer argumento a la clase en lugar de a una instancia, por lo que la convención es nombrar ese parámetro como `cls` en vez de `self`.

```
class Estudiante:
    def __init__(self, nombre, ingreso):
        self.nombre = nombre
        self.ingreso = ingreso

    @classmethod
    def desde_texto(cls, texto):
        nombre, ingreso = texto.split(",")
        return cls(nombre, int(ingreso))

    def resumen(self):
        return f"Estudiante(nombre={self.nombre}, ingreso={self.ingreso})"
```

Línea 6

La decoración `@classmethod` indica que el método `desde_texto` se invoca desde la clase y reciba a la clase como primer argumento.

Línea 7

Por convención, ese primer argumento se llama `cls`.

Línea 9

A partir de `cls`, se crea y devuelve una nueva instancia de la clase (en este caso, `Estudiante`) utilizando los valores requeridos por su método `__init__`.

Aunque exista exista un método de clase para crear objetos, se puede seguir creando objetos de la manera usual:

```
e1 = Estudiante("El Nombre", 2023)
e1.resumen()
```

```
'Estudiante(nombre=El Nombre, ingreso=2023)'
```

La diferencia es que ahora podemos usar el método `desde_texto` para crear un objeto `Estudiante` a partir de una cadena de texto con un formato específico.

```
e2 = Estudiante.desde_texto("El Estudiante, 2024")
e2.resumen()
```

```
'Estudiante(nombre=El Estudiante, ingreso=2024)'
```

Los atributos del objeto muestran los valores que esperamos en este caso.

```
e2.nombre, e2.ingreso
```

```
('El Estudiante', 2024)
```

Otro escenario en el que resulta útil usar métodos de clase es cuando queremos crear objetos "preconfigurados".

Por ejemplo, si tenemos una clase que representa sándwiches con una cantidad arbitraria de ingredientes, podemos definir métodos de clase que construyan instancias con combinaciones de ingredientes **preestablecidas**:

```
class Sandwich:
    def __init__(self, *ingredientes):
        self.ingredientes = ingredientes

    @classmethod
    def jyq(cls):
        return cls("jamón", "queso")

    @classmethod
    def mediterraneo(cls):
        return cls("tomate", "mozzarella", "rúcula", "aceitunas")
```

```
    def resumen(self):
        return f"Sandwich de: {', '.join(self.ingredientes)}"
```

```
s1 = Sandwich("tomate", "lechuga", "queso")
s1.resumen()
```

```
'Sandwich de: tomate, lechuga, queso'
```

```
Sandwich.jyq().resumen()
```

```
'Sandwich de: jamón, queso'
```

```
Sandwich.mediterraneo().resumen()
```

```
'Sandwich de: tomate, mozzarella, rúcula, aceitunas'
```

⚠ Atributos y métodos *a posteriori*

Python es tan flexible como lenguaje que incluso podemos asignar atributos y métodos luego de su definición.

```
class Rectangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

def f(self):
    return self.base * self.altura

Rectangulo.area = f
Rectangulo.atributo = "Algo"

r = Rectangulo(3, 2)
r.area()
# 6
```